Database Development and Design (SOE201)

Lecture 5b:
Query Evaluation Part 2

Dr. Wei Wang

AIAT

Learning Outcomes

- Algorithms for Evaluating Join
  - Nested Loop Join
  - Block Nested Loop Join
  - Indexed Nested Loop Join
  - Merge Join
  - Hash Join
- Algorithms for Evaluating Aggregation
- Algorithms for Evaluating Set Operations

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Natural-Join Operator

Notation: r ⋈ s

- Let r and s be relations on schemas R and S respectively. Then, r ⋈ s is relation on schema $R \cup S$ obtained as follows:
  - Consider each pair of tuples t_r from r and t_s from s.
  - If t_r and t_s have same value on each of attributes in $R \cap S$, add tuple t to result, where
    - t has same value as t_r on r
    - t has same value as t_s on s
- Example:

  $R = (A, B, C, D)$

  $S = (E, B, D)$
  - Result schema = (A, B, C, D, E)
  - $r \bowtie s$ is defined as:

$$\Pi_{r.A,\, r.B,\, r.C,\, r.D,\, s.E}(\sigma_{r.B = s.B \wedge r.D = s.D}(r \times s))$$

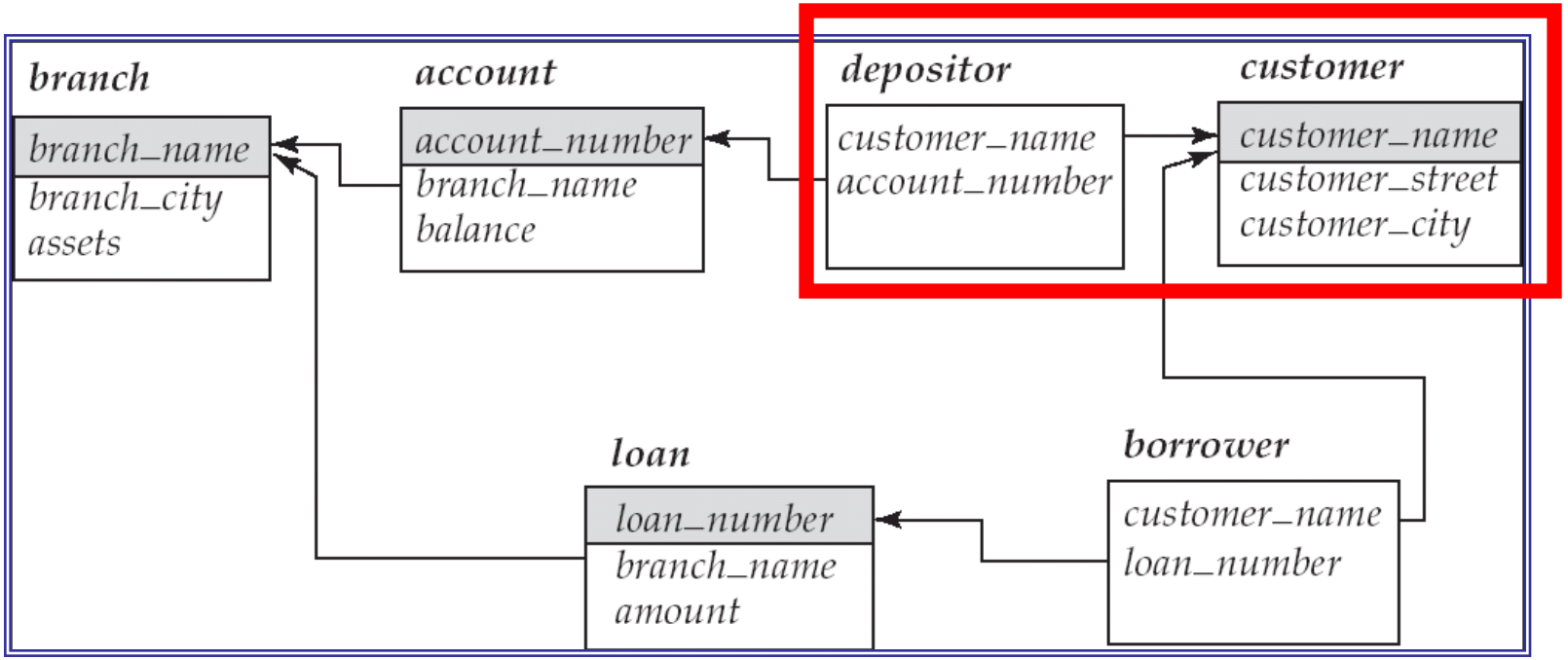
Xi'an Jiaotong-Liverpool University 西交利物浦大学

Join Operation

- Several different algorithms to implement joins exist
  - Nested-loop join
  - Block nested-loop join
  - Indexed nested-loop join
  - Merge-join
  - Hash-join
- Choice is based on *cost estimate*.

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Banking Example



- Examples in rest slides use following information:
  - Number of records of *customer*: 10,000
  - Number of blocks of *customer*: 400
  - Number of records of *depositor*: 5,000
  - Number of blocks of *depositor*: 100

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Nested-Loop Join

- Simple join algorithm that can be used independently of everything.
- To compute theta join: $r \bowtie_\theta s$

```
for each tuple t_r in r do begin
  for each tuple t_s in s do begin
      test pair (t_r, t_s) to see if they satisfy join condition θ
          if they do, add t_r • t_s to the result;
  end
end
```

- r is called **outer relation** and s **inner relation** of join.
- Quite expensive in general, since it requires to examine every pair of tuples in two relations.

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Nested-Loop Join cont'd

- *In **worst case**, if there is enough memory **only** to hold one block of each relation.*
  - *By default, worse scenarios are always considered unless explicitly stated.*
- Let n_r be number of tuples in relation r, b_r and b_s be number of blocks of *r* and *s*. Estimated cost is:
  - $n_r * b_s + b_r$ block transfers, and
  - $n_r + b_r$ seeks
- If smaller relation fits entirely in memory, use it as inner relation.
  - Reduces cost to $b_r + b_s$ block transfers and 2 seeks
- In general (but *not* always), it is better to have smaller relation as outer relation if none fits in memory.

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Nested-Loop Join Cost - Example

- Assuming worst case memory availability, cost estimate is
  - with *depositor* as outer relation:
    - 5,000 * 400 + 100 = 2,000,100 block transfers,
    - 5,000 + 100 = 5,100 seeks
  - with *customer* as outer relation
    - 10,000 * 100 + 400 = 1,000,400 block transfers,
    - 10,000 + 400 = 10,400 seeks
- If smaller relation (*depositor*) fits entirely in memory, cost estimate will be 500 block transfers and 2 seeks

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Block Nested-Loop Join

- Variant of nested-loop join in which every block of inner relation is paired with every block of outer relation.

```
for each block B_r of r do begin
    for each block B_s of s do begin
        for each tuple t_r in B_r do begin
            for each tuple t_s in B_s do begin
                Check if (t_r, t_s) satisfy join condition;
                    if they do, add t_r • t_s to result;
            end
        end
    end
end
```

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Block Nested-Loop Join Cost

- Worst case estimate:
  - $b_r * b_s + b_r$ block transfers, and
  - $2 * b_r$ seeks
  - Each block in inner relation s is read once for each block in outer relation (instead of once for each tuple in outer relation).
- If smaller relation fits into memory, cost is: $b_r + b_s$ block transfers and 2 seeks.

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Indexed Nested-Loop Join

- Index lookups can replace file scans if
  - Join is equi-join or natural join, and
  - Index is available on inner relation's join attribute
- To compute join

  For each tuple t_r in outer relation r

  *use **index** on inner relation s to look up tuples that satisfy join condition with tuple t_r*

  if they do, add $t_r \cdot t_s$ to result;
- In some cases, it pays to construct index just to compute join.

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Indexed Nested-Loop Join – cont'd

- Worst case: buffer has space for only one page of *r*.
- Cost of join: $b_r + n_r * c$ *block transfers and* $b_r + n_r * c$ *seeks*
  - Where *c* is cost of traversing index and fetching all matching *s* tuples for one tuple in *r*, e.g. $h_i + 1$ using B⁺-tree.
  - *c* can be estimated as cost of single selection on *s* using join condition
- If indices are available on join attributes of both *r* and *s*, use relation with fewer tuples as outer relation.

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Example of Indexed Nested-Loop Join Costs

- Compute *depositor* ⋈ *customer* using indexed nested loop join, with *depositor* as outer relation.
  - Suppose *customer* have primary B^+-tree index on join attribute *customer-name* (also key in this case), with n=20.
  - *customer* has 10,000 tuples, height of B^+-tree is at most 4, and one more access is needed to find actual data
  - Relation *depositor* has 5,000 tuples
- Nested loop join: 2,000,100 block transfers and 5,100 seeks
- Block nested loops join: 400*100 + 100 = 40,100 block transfers and 2 * 100 = 200 seeks
- Indexed nested loops join: 100 + 5,000 * (4+1) = 25,100 block transfers and seeks.
  - Number of block transfers is less than that for block nested loop
  - But number of seeks is much larger
  - In this case, using Indexed Nested Loop Join doesn't pay

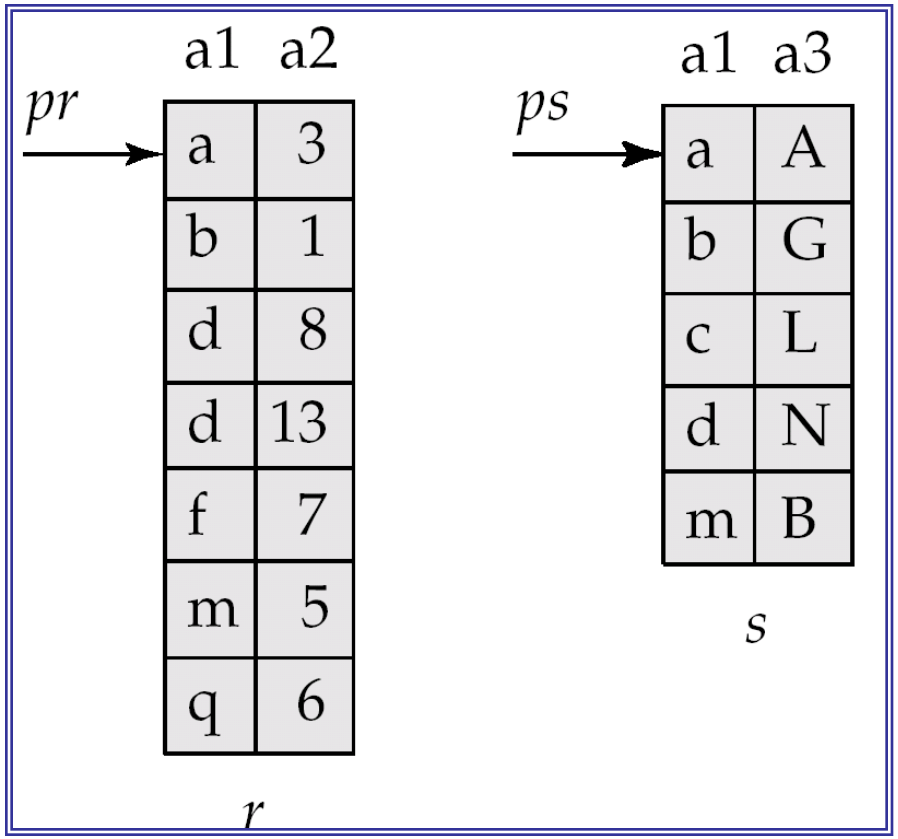
Xi'an Jiaotong-Liverpool University 西交利物浦大学

Merge-Join

- Merge join needs two steps:
  - Sort both relations on join attribute, if not already sorted;
  - Join step is similar to merge stage of external sort-merge.
- Merge-join algorithm
  - Initialise two pointers, p_r and p_s, pointing to r and s
  - While not done
    - p_r and p_s move through relations.
    - Group of tuples of inner relation s with same value on join attributes is read into set S_s.
    - Do join on tuple pointed to by p_r and tuples in S_s
  - End while

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Merge-Join cont'd



Xi'an Jiaotong-Liverpool University
西交利物浦大学

Merge-Join cont'd

- Only for equi-joins and natural joins
- Each block needs to be read only once
  - *Assuming that all tuples for any given value of join attributes fit in memory*
- Thus, cost of merge join is:

  $b_r + b_s$ block transfers +

  $\lceil b_r / b_b \rceil + \lceil b_s / b_b \rceil$ seeks
  - ***Plus cost of sorting if relations are unsorted.***
  - b_b is number of blocks allocated in memory for each relation
  - It makes sense to allocate multiple buffer blocks to each relation, provided extra memory is available.

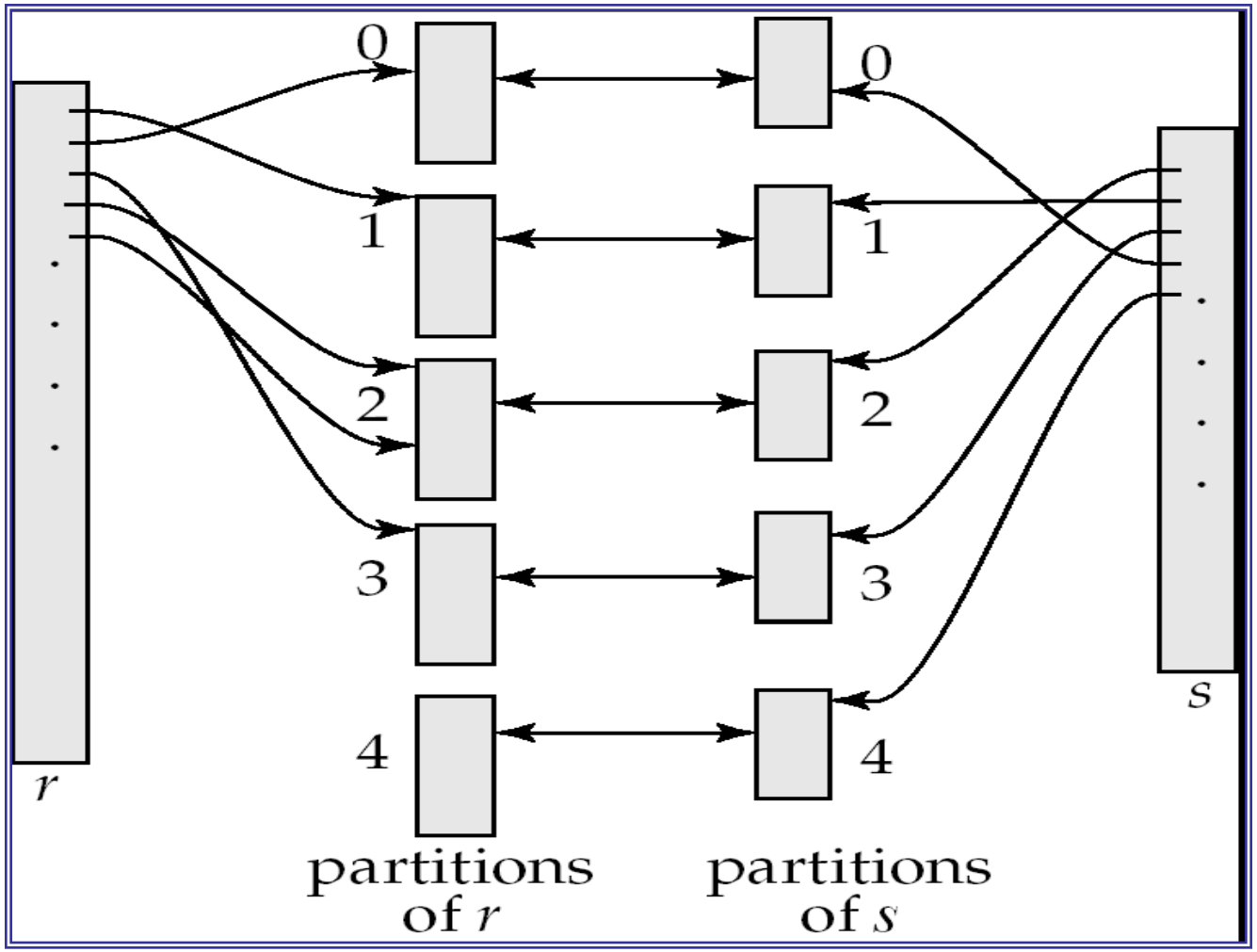
Xi'an Jiaotong-Liverpool University 西交利物浦大学

Hash-Join

- Only applicable for equi-joins and natural joins.
- Hash function *h* is used to partition tuples of both relations
- *h* maps *JoinAttrs* values to patitions {0, 1, ..., *n*}, where *JoinAttrs* denotes common attributes of *r* and *s*.
  - $r_0, r_1, \ldots, r_n$ denote partitions of *r* tuples
    - Each tuple $t_r \in r$ is put in partition r_i where $i = h(t_r[JoinAttrs])$.
  - $s_0, s_1, \ldots, s_n$ denotes partitions of *s* tuples
    - Each tuple $t_s \in s$ is put in partition s_i, where $i = h(t_s[JoinAttrs])$.
- General idea:
  - Partition relations according to above
  - Perform join on each partitions r_i and s_i
    - No need to compute join between different partitions
    - If value is hashed to *i*, *r* tuple having that value is in r_i and *s* tuple having that value is in s_i.

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Hash-Join cont'd



Xi'an Jiaotong-Liverpool University 西交利物浦大学

Hash-Join Algorithm

1. Partition relation *s* using hashing function *h*. When partitioning relation, some blocks of memory (b_b) are reserved for input buffer and output buffer for each partition;
2. Partition *r* similarly;
3. For each partition *i*:
   (a) Load s_i into memory and build in-memory hash index using join attribute. This hash index uses ***different hash*** function than earlier *h* for partitioning.
   (b) Read tuples in r_i from disk. For each tuple t_r, locate each matching tuple t_s in s_i using in-memory hash index. Output concatenation of their attributes.

*Relation s is called **build input** and r is called **probe input**.*

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Hash-Join Algorithm cont'd

- Number of partitions *n* for hash function *h* is chosen such that *each s_i should fit in memory*.
  - Typically *n* is chosen as $\lceil b_s/M \rceil * f$, where *f* is "fudge factor", typically around *1.2*, to avoid overflows
  - Probe relation partitions r_i need not fit in memory

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Cost of Hash-Join

- Cost of hash join:
  - $3(b_r + b_s) + 4 * n_h$ block transfers
    - Or, $2(b_r + b_s) + 2 * n_h + (b_r + b_s) + 2 * n_h$
  - $2(\lceil b_r / b_b \rceil + \lceil b_s / b_b \rceil) + 2 * n_h$ seeks
  - Each of n_h partitions could have partially filled block that has to be written and read back
  - Build and probe phases require only one seek for each of n_h partitions of each relation, since each partition can be read sequentially.
- If entire build input can be kept in main memory (then no partitioning is required), cost estimate goes down to $b_r + b_s$ *and 2 seeks*.

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Cost of Hash-Join Example

- Assume that memory size is 20 blocks, $b_{depositor}$= 100 and $b_{customer}$ = 400.
  - Assume relation *depositor* is used as build input. Partition it into five partitions, each of size 20 blocks.
  - Similarly, partition *customer* into five partitions, each of size 80.
- Assuming b_b=3 blocks are allocated for input buffer and output buffer
- Therefore total cost:

  3(100 + 400) + 4*5= 1, 520 block transfers +
  2(⌈100/3⌉ + ⌈400/3⌉) + 2*5 = 346 seeks
- Compare to previous algorithms:
  - Block nested loop: 40,100 block transfers and 200 seeks;
  - Index nested loop: 25,100 block transfers and 25,100 seeks.

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Aggregation

- **Aggregation** can be implemented similarly to duplicate elimination.
  - Sorting or hashing can be used to bring tuples into same group together, and then aggregate functions can be applied on each group.
  - *Optimisation:* combine tuples in same group during run generation and intermediate merges, by computing partial aggregate values
    - For count, min, max, sum: keep aggregate values on tuples found so far in group.
    - For avg, keep sum and count, and divide sum by count.

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Set Operations

- Set operations ($\cup$, $\cap$ and -): can either use variant of merge-join after sorting, or variant of hash-join.
- Set operations using hashing:
  - Partition both relations using same hash function
  - Process each partition i as follows.
    - Using different hashing function, build in-memory hash index on r_i.
    - Process s_i as follows
      - $r \cup s$
        - Add tuples in s_i to hash index if they are not in.
        - At end, add tuples in hash index to result.
      - $r \cap s$
        - Output tuples in s_i to result if they are already in hash index
      - $r - s$
        - For each tuple in s_i, if it is in hash index, delete it from index.
        - At end, add remaining tuples in hash index to result.

Xi'an Jiaotong-Liverpool University 西交利物浦大学

End of Lecture

- Reading
  - Textbook Chapters 15.5 and 15.6